



JavaScript Fundamentals & the DOM

Web Design and Programming

M.Sc. Le Duy Khanh

Faculty of Data Science and Artificial Intelligence



Session Outline

- 1. Adding JavaScript to a Page**
- 2. JS Fundamentals: Python → JavaScript**
- 3. The Document Object Model (DOM)**
- 4. Events & Client-Side Form Validation**
- 5. Review & Summary**



Learning Objectives

By the end of this session, you will be able to:

1. Embed JavaScript in a page with inline and external `<script>` tags, placed correctly.
2. Write foundational JS syntax by mapping it directly onto Python you already know.
3. Use the DOM to select, read, and modify HTML elements from JavaScript.
4. Create and remove elements dynamically with `createElement()/appendChild()/removeChild()`.
5. Attach events with `addEventListener()` and validate a form on the client side.



How to Approach This Session

- This is the **heaviest content session** of the course – the “pure JS” day.
- Constant mental habit: **Python** → **JavaScript**. Same logic, different punctuation.
- The **DOM** is just a tree – think of it as a nested JSON/dict that the browser renders visually.
- End goal for today: **client-side form validation** – catching bad input before it ever reaches a server.



Environment Check

- Open **VS Code** with the **Live Server** extension running.
- Create app.js and link it at the end of <body> in your Week 2 HTML page: `<script src="app.js" defer></script>`
- Open the browser **DevTools Console** – this is where you will read output and errors all session.



Adding JavaScript to a Page

Embedded vs. External Scripts

```
1 <!-- Embedded: JS written directly inside the page -->
2 <script>
3     alert("Hello from an embedded script");
4 </script>
5
6 <!-- External: JS kept in its own file -->
7 <script src="app.js"></script>
```

- External scripts are reusable across pages – same rationale as external CSS.
- Analogy: like separating a .py module from a notebook for maintainability.

Script Placement – Avoid a Common Bug

```
1 <body>
2   ...page content...
3   <script src="app.js" defer></script>
4 </body>
```

- Place scripts just before `</body>`, or use the `defer` attribute.
- **Why:** if a script runs before the DOM is built, `getElementById(...)` fails to find the element.
- **Exam cue:** “element not found” errors are usually a script-placement problem.



JS Fundamentals: Python → JavaScript

Variable Declaration

```
1 let bedrooms = 3;    // can be reassigned later
2 const taxRate = 0.08; // cannot be reassigned
```

Python	JavaScript	Notes
x = 5	let x = 5;	reassignable
-	const x = 5;	cannot reassign
-	var x = 5;	legacy, avoid

JS statements end with ;; blocks use { } instead of indentation.

Data Types

```
1 let price;           // undefined - declared, no value yet
2 let result = null;   // explicitly "no value"
3 let sqft = 1200;     // Number
4 let city = "Boston"; // String
5 let isSold = true;   // Boolean
```

Python	JavaScript
None	undefined / null
int/float	Number
str	String
bool	Boolean
list / dict	Array / Object

Comparison Operators – Trickiest Exam Topic

```
1 "10" == 10    // true  - values compared after type coercion
2 "10" === 10   // false - values AND types must match
3 "10" != 10    // false
4 "10" !== 10   // true
```

- **Rule of thumb:** always use `===` / `!==`, never `==` / `!=`.
- **DS/AI analogy:** like checking `dtype` in pandas – `"5" == 5` silently ignores that one is a string and one is an int.

Control Structures

```
1 if (bedrooms > 3) {  
2     console.log("Large house");  
3 } else {  
4     console.log("Standard house");  
5 }  
6  
7 for (let i = 0; i < 5; i++) {  
8     console.log(i);  
9 }
```

- Same logic as Python's if/elif/else and for/while.
- Only the punctuation changes: no :, no indentation-as-syntax, curly braces mark blocks.

Functions

```
1 // Function declaration - equivalent to Python's def
2 function square(x) {
3     return x * x;
4 }
5
6 // Arrow function - equivalent to a Python lambda
7 const double = (x) => x * 2;
```

Python	JavaScript
def greet(name):	function greet(name) { }
lambda x: x * 2	(x) => x * 2

Arrays and Objects

```
1 // Array - equivalent to a Python list
2 let scores = [88, 92, 79];
3
4 // Object literal - equivalent to a Python dict
5 let sample = { id: 1, name: "House A", predicted_price: 452000 };
6
7 scores[0];           // 88
8 sample.name;         // "House A"
```

- Array $\approx$ list; Object literal $\approx$ dict.
- These two structures are exactly what JSON data (Week 5) is built from.



Further Reading (Part 1)

Optional – self-study handout

- Variable scope: `let/const` are block-scoped; `var` is function-scoped.
- Template literals: `'Hello ${name}'`.
- Source: *Learning Web Design*, Ch. 21 – Introduction to JavaScript.



Lab 1: Core JS Syntax on Array/Object Data

Hands-on practice

Model a list of prediction samples as `[{id, name, result}, ...]`:

- Write a for loop that filters the array by a condition.
- Write a function that sums a numeric field across all objects.
- Write a function that finds the object with the largest value of a field, using a for loop.
- *Finished early?* Convert one function to an arrow function.



The Document Object Model (DOM)



The Node Tree

- The browser turns every HTML document into a tree of **nodes** that JavaScript can read and modify.
- Each tag, attribute, and even text content is a node in this tree.
- **Mental model for DS/AI:** the DOM is a nested JSON/dict structure the browser happens to render on screen.

Selecting Elements

```
1 document.getElementById("beginner");           // one element, by id
2 document.getElementsByClassName("card");        // a collection, by class
3 document.querySelector(".card");               // first match, CSS selector
4 document.querySelectorAll(".dashboard p");     // all matches, CSS selector
```

- `querySelector()/querySelectorAll()` are recommended – same selector syntax as Week 3 CSS.
- They express far more specific targets: `input[type="text"]`, `.dashboard .card`, etc.

Reading and Modifying Content

```
1 let card = document.querySelector(".card");
2
3 card.innerHTML;           // get/set HTML content
4 card.textContent;         // get/set plain text
5 card.getAttribute("data-id"); // read an attribute
6 card.setAttribute("data-id", "42"); // write an attribute
7 card.style.color = "red";    // set a CSS property directly
```

- Hyphenated CSS properties become camelCase: background-color → style.backgroundColor.

Creating and Removing Elements

```
1 // Create a new element and text, then attach it to the page
2 let errorMsg = document.createElement("p");
3 errorMsg.textContent = "This field is required.";
4 errorMsg.style.color = "red";
5
6 let form = document.querySelector("#house-form");
7 form.appendChild(errorMsg);
8
9 // Remove an element later
10 form.removeChild(errorMsg);
```

- This is how JS “draws” new content onto a page without a full reload.
- Same technique used in Week 5 to render API results into the page.

Further Reading (Part 2)

Optional – self-study handout

- `insertBefore()`, `replaceChild()` – more precise node placement.
- `nodeLists` behave like arrays but are “live” collections – mutate carefully in a loop.
- Source: *Learning Web Design*, Ch. 22 – Using JavaScript and the DOM.



Events & Client-Side Form Validation

The Event Mechanism

```
1 let btn = document.querySelector("#submit-btn");  
2 btn.addEventListener("click", function (event) {  
3     console.log("Button clicked!");  
4     console.log(event.target); // the element that triggered the event  
5 });
```

- `addEventListener()` is the modern, recommended way to bind events.
- Keeps behavior out of HTML markup and allows **multiple** listeners per element – unlike `onclick="..."` or `element.onclick = fn`.

Client-Side Form Validation

```
1 let form = document.querySelector("#house-form");
2
3 form.addEventListener("submit", function (event) {
4     event.preventDefault(); // stop the default page reload
5
6     let bedrooms = document.querySelector("#bedrooms").value;
7
8     if (bedrooms === "" || Number(bedrooms) <= 0) {
9         let error = document.createElement("p");
10        error.textContent = "Bedrooms must be a positive number.";
11        form.appendChild(error);
12    } else {
13        console.log("Form is valid - ready to send.");
14    }
15 });
```



Why Validate on the Client?

- `event.preventDefault()` inside a submit handler blocks the default reload/request.
- Errors are shown **immediately** – no round trip to the server needed.
- **DS/AI analogy:** validating input before it reaches a backend prediction API is the same instinct as validating a DataFrame's dtypes/ranges before feeding it to a model – just one layer earlier in the pipeline.

Further Reading (Part 3)

Optional – self-study handout

- Common events: change, keyup, focus, blur.
- Built-in HTML5 validation attributes: required, pattern, min/max.
- Source: MDN – *EventTarget.addEventListener()*; *Client-side form validation*.

Lab 2: DOM + Events on the Week 2 Form

Hands-on practice

Reuse the house-price input form from Week 2:

- Attach `addEventListener("submit", ...)` and call `event.preventDefault()`.
- Check required fields are not empty, and "bedrooms" is a positive number.
- On failure: create a red `<p>` error with `createElement()/appendChild()`.
- On success: display a "Ready to submit" message.
- *Finished early?* Clear the previous error before showing a new one.



Review & Summary

Quick MCQ Review

1. Result of "10" == 10? And "10" === 10? **true, false**
2. Difference between let and const? **const cannot be reassigned**
3. DOM method for CSS-style selectors? **querySelector()/querySelectorAll()**
4. Recommended way to bind multiple handlers to one element?
addEventListener()
5. Statement that blocks a form's default submit/reload? **event.preventDefault()**
6. Default value of a declared-but-unassigned variable? **undefined**



Self-Check Checklist

- ☐ Script linked via `<script src="app.js" defer></script>`
- ☐ Always use `===/!==` instead of `==/!=`
- ☐ Comfortable with `querySelector()/querySelectorAll()`
- ☐ Can create, append, and remove a DOM element from a script
- ☐ Form submit blocked with `event.preventDefault()` and validated
- ☐ Error message displayed dynamically, not via `alert()` only



Summary and Next Session

- **JavaScript syntax** maps closely to Python – variables, types, control flow, functions, arrays/objects.
- **The DOM** represents the page as a tree of nodes; JS selects, reads, writes, creates, and removes nodes.
- **Events**, bound with `addEventListener()`, let JS react to user actions.
- **Client-side form validation** combines `submit` + `event.preventDefault()` for instant feedback.
- Next: **JSON and APIs** – fetching real data and rendering it into the DOM (Week 5).

Questions & Answers